Guía de preparación para la auditoría presencial

Hotel Reservas · Sistemas Transaccionales · Grupo A1 · Gustavo Adolfo Cardona Bayona, Jonathan Soler Guevara, Hugo Andrés Forero Angel · Docente: Jamilton Fernando Benvides

1. Lista de verificación para mañana

- ☐ **Formato de auditoría impreso**: archivo `Ficha de Auditoria Presencial - Hotel Reservas.pdf` (una copia por si el docente quiere diligenciar la suya y otra de respaldo). Llevar también impresa la `Ficha Requerimientos del Sistema`.
- ☐ **Presentación de trazabilidad**: abrir `Presentacion Trazabilidad Documento Maestro - Hotel Reservas.html` en Chrome o Edge y pulsar F11 (pantalla completa); avanzar con las flechas. Copia de seguridad en PDF del mismo nombre.
- ☐ **Prototipo interactivo**: abrir `Prototipo Interactivo - Hotel Reservas.html` en el navegador (funciona sin internet). Si se dispone del emulador Android con Visual Studio, tener también abierta la app .NET MAUI del repositorio para mostrar la pantalla de inicio real.
- ☐ **SQL Server** (opcional pero muy valorado): si SQL Server y SSMS están instalados, ejecutar la noche anterior los scripts 01, 02 y 03 del repositorio y tener abiertas dos ventanas con `04_demo_concurrencia_auditoria.sql`.
- ☐ **Documento maestro**: abierto en PDF (61 páginas) para responder preguntas señalando la sección. Repositorio abierto: github.com/gustavo8015/HotelReservas.
- ☐ Cargador del portátil, adaptador HDMI, y el documento también en una memoria USB.

2. Guion de la presentación de trazabilidad (5 minutos)

Siete diapositivas, un expositor principal por bloque y los demás listos para complementar. Practicar con cronómetro: el objetivo es terminar en 4 minutos 30 segundos y dejar margen.

0:00 – 0:30 **Diapositiva 1, portada.** Gustavo
"Buenas tardes. Somos el grupo A1 y presentamos Hotel Reservas, un sistema transaccional móvil para la gestión de reservas de habitaciones. En cinco minutos vamos a mostrar cómo el documento maestro creció desde la contextualización hasta la especificación y cómo cada elemento del diseño se puede rastrear hasta el problema."

0:30 – 1:30 **Diapositiva 2, cómo creció el documento.** Gustavo
"En la Fase I describimos el problema sin tecnología: hoteles pequeños que llevan las reservas en cuadernos y chats, con sobreventas, reservas perdidas, diferencias entre reserva y pago, y sin evidencia. En la Fase II tomamos las decisiones de diseño y las justificamos una a una: arquitectura en capas, 15 casos de uso, clases, MER y modelo relacional. En la Fase III especificamos qué debe hacer el sistema y bajo qué condiciones: 15 requerimientos funcionales, 5 no funcionales, los marcos y el servicio de IA. Abajo se ve cómo incorporamos la retroalimentación del docente en cada fase."

1:30 – 2:30 **Diapositiva 3, la cadena de trazabilidad.** Jonathan
"Esta es la cadena completa con el problema principal: reservas dobles. Lleva al objetivo específico de un modelo relacional con bloqueo, a los casos de uso CU-03 y CU-04, a los requerimientos RF03, RF04 y RNF02, a las clases ServicioReservas y Reserva, a las tablas RESERVA y HABITACION, y al componente que lo resuelve: el gestor transaccional con SERIALIZABLE y UPDLOCK en SQL Server. La matriz muestra lo mismo para los seis problemas identificados."

2:30 – 3:15 **Diapositiva 4, del problema al diseño.** Jonathan
"Casos de uso con include y extend; diagrama de clases con Reserva al centro y Pago, Comprobante y Bitácora por composición; modelo relacional con nueve tablas. Las cardinalidades son las mismas en los tres artefactos, que fue una de las correcciones de la Fase II."

3:15 – 3:50 **Diapositiva 5, arquitectura y base de datos.** Hugo
"Cuatro capas: .NET MAUI, ASP.NET Core, Entity Framework Core y SQL Server. La app nunca toca la base de datos. La transacción crítica verifica la disponibilidad con bloqueo y registra reserva, pago, comprobante y bitácora en una sola unidad de trabajo; si algo falla, ROLLBACK."

3:50 – 4:25 **Diapositiva 6, Fase III.** Hugo
"Los 15 RF corresponden uno a uno a los casos de uso y los 5 RNF tienen criterio verificable; el más importante, RNF02: con dos sesiones sobre la última habitación, una confirma y la otra recibe HTTP 409. El asistente de IA se describe por cinco componentes y solo accede a funciones de lectura."

4:25 – 4:50 **Diapositiva 7, prototipo y estado.** Gustavo
"Esto es lo hecho, lo pendiente y el repositorio. Pasamos a la demostración."

3. Guion de la demostración del prototipo (5 minutos)

Qué decir antes de empezar: "El prototipo tiene dos partes: la aplicación .NET MAUI del repositorio con la pantalla de inicio y la navegación, y este prototipo interactivo de las funciones transaccionales, que simula la API y la base de datos con los mismos datos del script 03. Sirve para demostrar el comportamiento diseñado; la API real es el siguiente entregable." Ser

transparentes evita que el docente considere que se presenta como terminado algo que no lo está.

0:00 – 0:40 **Inicio.** Hugo Mostrar los indicadores (ocupadas, disponibles, reservadas) y los módulos. Señalar el panel derecho: "a la derecha se ve lo que ocurre en la API y en SQL Server con cada acción".

0:40 – 1:30 **CU-03 Consultar disponibilidad.** Tipo Doble, 2 personas, 16 al 18 de octubre, Buscar. Explicar: la 204 aparece reservada porque ya hay una reserva confirmada en esas fechas y la 302 en mantenimiento; el resultado lo calcula el servidor con el índice sobre habitación y fechas. Probar una fecha de salida anterior a la de entrada para mostrar la validación (restricción CHECK).

1:30 – 2:30 **CU-04 Confirmar reserva, camino feliz.** Tocar la habitación 103, elegir huésped Laura Martínez, medio Tarjeta, Confirmar. Leer el registro: SERIALIZABLE, BEGIN TRANSACTION, SELECT con UPDLOCK y HOLDLOCK, INSERT en RESERVA, PAGO, COMPROBANTE y BITACORA, COMMIT. Mostrar que los cuatro contadores subieron a la vez y la fila nueva en la tabla RESERVA con su version_fila.

2:30 – 3:40 **RNF02 Prueba de concurrencia.** Jonathan Volver a Reservas, buscar Doble 16 al 18, tocar la 203. En el panel derecho pulsar **Activar** en "Prueba de concurrencia" y luego Confirmar. Explicar lo que muestra el registro: otra sesión ganó el bloqueo y confirmó la 203; nuestra verificación encontró el solapamiento, hizo ROLLBACK y lanzó el error 50001, que la API traduce a HTTP 409. Subrayar: el contador de conflictos sube, pero el de pagos y comprobantes no; no quedan datos parciales. Si hay SQL Server disponible, aquí se ejecuta el script 04 en dos ventanas de SSMS para mostrarlo real.

3:40 – 4:20 **Historial y bitácora (RNF05).** Abrir Historial: la reserva recién creada con su código; registrar el check-in y mostrar que la bitácora anota usuario, fecha y cambio de estado, y que la version_fila cambia (concurrencia optimista).

4:20 – 5:00 **Asistente de IA.** Gustavo Usar los chips: "Dobles libres 16-18 oct" (responde con datos reales y cita la función consultada) y "Reserva la 103" (se niega y remite a la pantalla de reservas). Cerrar: "el asistente consulta, nunca transacciona; la integridad no depende del modelo".

4. Reparto de temas por integrante

El docente preguntará a todos. Cada integrante debe dominar todo el proyecto, pero conviene tener un responsable principal por tema para responder primero y que los demás complementen.

Gustavo: problemática y necesidad (Fase I), objetivos, marco contextual, servicio de IA y sus cinco componentes, estado del repositorio y plan de continuidad.

Jonathan: casos de uso (include/extend, flujos de CU-04 y CU-08), diagrama de clases y multiplicidades, MER y modelo relacional, matriz de trazabilidad, prueba de concurrencia.

Hugo: arquitectura en capas y justificación tecnológica, arquitectura de la base de datos, transacción y procedimiento almacenado, requerimientos funcionales y no funcionales, prototipo y app MAUI.

Todos: saber explicar ACID con el ejemplo de la reserva, qué es la sobreventa, por qué la app no accede a la base de datos y qué queda pendiente.

5. Banco de preguntas y respuestas

Problema, objetivos y alcance

> **¿Cuál es el problema que resuelve el sistema?**
> Los hoteles pequeños y medianos gestionan las reservas con cuadernos, hojas compartidas y mensajería; la información queda dispersa entre turnos y no hay un registro único. Eso produce sobreventas (dos reservas para la misma habitación y fechas), reservas perdidas, diferencias entre lo reservado, lo pagado y lo facturado, y falta de evidencia para atender reclamaciones.

> **¿Cuál es el objetivo central?**
> Diseñar una aplicación móvil transaccional que asegure integridad, coherencia y trazabilidad de las reservas, de modo que las operaciones simultáneas de confirmar, modificar y cancelar se procesen con propiedades ACID sobre una fuente central de disponibilidad.

> **¿Qué queda dentro y fuera del alcance?**
> Dentro: un hotel, inventario central, dos perfiles (recepcionista y administrador), una habitación por reserva, ciclo completo de la reserva y reportes básicos de ocupación. Fuera: pasarelas de pago, gestores de canales, contabilidad externa, reserva en línea por el huésped y varios hoteles. El asistente de IA se documenta y se diseña, no se implementa en esta fase.

> **¿Por qué una app móvil (APK) y no una web?**
> Porque el personal se desplaza por el hotel y necesita consultar y confirmar desde donde esté; además fue la modalidad recomendada por el docente. La arquitectura en capas permite agregar un cliente web después sin cambiar la API.

> **¿Qué diferencia hay entre las tres fases?**
> Fase I: contextualización sin decisiones técnicas. Fase II: análisis y diseño, con cada decisión justificada frente al problema. Fase III: especificación de requerimientos, marcos conceptual, contextual e integrador, arquitectura de la base de datos y prototipo. La implementación de la API y la app operativa pertenece a las etapas siguientes.

Sistemas transaccionales y concurrencia

¿Qué es una transacción y qué significa ACID en este proyecto?

Una transacción es una unidad lógica de trabajo que se ejecuta completa o no deja efecto. Atomicidad: reserva, pago, comprobante y bitácora se guardan juntos o ninguno. Consistencia: no puede quedar una reserva que viole las restricciones (fechas, solapamiento, claves). Aislamiento: dos recepcionistas no ven los estados intermedios del otro. Durabilidad: una vez confirmada, la reserva sobrevive a un fallo gracias al registro de transacciones.

¿Cómo se evita la reserva doble?

Dentro de la transacción de confirmación, con aislamiento SERIALIZABLE, se consulta RESERVA con las sugerencias UPDLOCK y HOLDLOCK sobre las filas de esa habitación en esas fechas. La segunda sesión queda bloqueada hasta que la primera confirma; al continuar, encuentra el solapamiento, hace ROLLBACK y lanza el error 50001, que la API convierte en HTTP 409. Además hay un índice sobre (id_habitacion, fecha_entrada, fecha_salida) y la columna ROWVERSION para concurrencia optimista en las actualizaciones.

¿Qué diferencia hay entre bloqueo pesimista y concurrencia optimista, y cuál usan?

Pesimista: se bloquea el recurso antes de usarlo (UPDLOCK/HOLDLOCK en la confirmación). Optimista: se lee una versión y la actualización solo se acepta si la versión no cambió (ROWVERSION en modificar, cancelar, check-in). Usamos pesimista para confirmar, porque la contención es alta y el costo de un error es una sobreventa, y optimista para los cambios de estado. La decisión definitiva se validará con pruebas en la etapa de construcción.

¿Qué es el nivel de aislamiento SERIALIZABLE y por qué no uno menor?

Es el nivel más estricto: el resultado equivale a ejecutar las transacciones una tras otra. Niveles menores como READ COMMITTED permiten que entre la verificación y la inserción otra sesión inserte una reserva solapada (lectura fantasma); SERIALIZABLE, junto con HOLDLOCK, bloquea el rango y lo impide.

¿Qué pasa si falla la inserción del comprobante a mitad de la transacción?

Con SET XACT_ABORT ON y el bloque TRY/CATCH, cualquier error provoca ROLLBACK de todo: no queda reserva ni pago sin comprobante. Es la atomicidad en la práctica.

¿Qué es OLTP?

Procesamiento de transacciones en línea: muchas operaciones cortas y concurrentes sobre una base de datos compartida, registradas en el momento en que ocurren. La recepción de un hotel es un caso típico.

Arquitectura y tecnologías

Describa la arquitectura.

Cliente-servidor en cuatro capas: presentación (app Android en C# con .NET MAUI, patrón MVVM), lógica de negocio (API REST en ASP.NET Core con controladores, servicios, gestor transaccional y seguridad JWT), acceso a datos (Entity Framework Core y ADO.NET para las sugerencias de bloqueo) y almacenamiento (Microsoft SQL Server). Cada capa habla solo con la inmediatamente inferior.

¿Por qué la app no se conecta directamente a la base de datos?

Porque el control transaccional y la seguridad deben estar en un solo punto: la API. Si cada teléfono accediera a la base de datos, habría que exponer credenciales en el dispositivo y no se podría garantizar el mismo comportamiento de bloqueo. Además permite cambiar la app o agregar un portal web sin tocar las reglas.

¿Por qué SQL Server y no MySQL, PostgreSQL o SQLite?
SQLite se descartó porque es embebido en el dispositivo y rompería el registro único (cada teléfono tendría su copia). MySQL se descartó por decisión del equipo. PostgreSQL es una alternativa equivalente sin ventaja decisiva para este alcance. SQL Server ofrece transacciones explícitas, niveles de aislamiento configurables, bloqueos de fila, ROWVERSION y se integra de forma nativa con .NET; además tiene ediciones gratuitas (Express y Developer).

¿Qué hace cada capa cuando se confirma una reserva?
La app valida el formato y envía POST /api/reservas con el token JWT. El controlador llama al servicio de reservas; el gestor transaccional abre la transacción y ejecuta el procedimiento sp_ConfirmarReserva a través de la capa de datos; SQL Server verifica con bloqueo, inserta en las cuatro tablas y confirma; la API responde 201 con el código de reserva y el comprobante, o 409 si hubo conflicto.

¿Qué es REST y qué es JWT?
REST es el estilo en el que cada recurso (reservas, habitaciones, huéspedes) tiene una ruta y se opera con GET, POST, PUT y DELETE intercambiando JSON sobre HTTPS. JWT es un token firmado que lleva la identidad y el perfil del usuario; la app lo envía en cada solicitud y la API lo valida sin consultar la base de datos.

¿Qué es MVVM?
Modelo, Vista y Modelo de Vista: las pantallas XAML (vista) se enlazan a modelos de vista que manejan el estado y las validaciones de formato, y estos llaman al cliente HTTP. Microsoft lo recomienda para .NET MAUI y separa la interfaz de la lógica de presentación.

Casos de uso, clases y datos

¿Cuántos casos de uso hay y qué relaciones include y extend tienen?
Quince, repartidos entre recepcionista y administrador. CU-04 Realizar reserva y CU-07 Modificar reserva incluyen CU-03 Consultar disponibilidad; CU-04 incluye CU-05 Registrar pago y CU-06 Generar comprobante porque son parte de la misma transacción; CU-15 Aplicar penalidad extiende CU-08 Cancelar reserva solo cuando la cancelación es tardía.

¿Por qué Usuario es abstracta y por qué existe ServicioReservas?
Usuario reúne autenticación e identidad y se especializa en Recepcionista y Administrador, que tienen permisos distintos. ServicioReservas no es una entidad del dominio sino la clase de la lógica de negocio que coordina la confirmación; se incluye porque es la que materializa CU-04.

¿Qué significan la composición y la agregación en el diagrama de clases?
Composición (rombo lleno): Pago, Comprobante y Bitácora no existen sin su Reserva. Agregación (rombo vacío): las habitaciones pertenecen a un tipo, pero pueden existir aunque se reorganice la clasificación.

¿Cuáles son las cardinalidades principales y dónde están las claves foráneas?
Tipo de habitación a habitaciones y a tarifas: 1 a 1..N. Huésped, habitación y usuario a reserva: 1 a 0..N. Reserva a pagos y a bitácora: 1 a 1..N. Reserva a comprobante: 1 a 0..1 (UNIQUE sobre id_reserva). Usuario a bitácora: 1 a 0..N. La clave foránea va siempre en la tabla del lado N; son iguales en el diagrama de clases, en el MER y en el modelo relacional.

¿Qué diferencia hay entre el MER y el modelo relacional?
El MER es conceptual: entidades, relaciones, atributos y cardinalidades, sin tipos de datos ni claves foráneas. El modelo relacional es lógico: tablas con columnas tipadas, claves primarias y foráneas, restricciones UNIQUE y CHECK. El segundo se deriva del primero.

¿Qué guarda la bitácora y para qué sirve?
Cada cambio de estado de una reserva con el usuario responsable, la fecha, el estado anterior y el nuevo. Es la evidencia para reclamaciones y cumple el requerimiento RNF05 de trazabilidad.

¿Qué es version_fila (ROWVERSION)?
Una columna que SQL Server actualiza automáticamente en cada modificación de la fila. La app envía la versión que leyó y la actualización solo se aplica si coincide; si otra sesión modificó la reserva, se rechaza. Es el control de concurrencia optimista.

¿Qué índices tiene la base de datos y por qué?
IX_RESERVA_habitacion_fechas sobre (id_habitacion, fecha_entrada, fecha_salida) acelera la consulta de disponibilidad y la verificación de solapamiento; además los UNIQUE de codigo, numero de comprobante, numero de habitación, documento del huésped y correo del usuario.

Requerimientos

¿Cómo se relacionan los RF con los casos de uso?
Uno a uno: RF01 a RF15 corresponden a CU-01 a CU-15, con actor y resultado esperado. Así no hay funciones sin respaldo en el diseño.

¿Por qué solo cinco RNF y cuáles son?
Por indicación del docente se formulan los determinantes: seguridad (JWT, perfiles, contraseñas con hash), integridad (ACID y prueba de dos sesiones con HTTP 409), rendimiento (disponibilidad menos de 2 s y confirmación menos de 3 s en el 95 % de los casos), usabilidad (reserva completa en máximo 5 pantallas y 3 minutos) y trazabilidad (historial completo desde la bitácora). Disponibilidad, mantenibilidad y compatibilidad quedan cubiertas por la arquitectura.

¿Cómo se verificará el RNF02?
Con dos sesiones simultáneas intentando la última habitación disponible: exactamente una confirma y la otra recibe 409; después se comprueba que no hay pagos ni comprobantes sin reserva. El script 04 del repositorio lo reproduce en SSMS.

Servicio de inteligencia artificial

¿Qué servicio de IA proponen y para qué?
Un asistente de consultas en lenguaje natural basado en Azure OpenAI Service (modelo GPT-4o con llamada a funciones). El recepcionista pregunta, por ejemplo, qué dobles hay libres del 16 al 18; el modelo identifica la función y los parámetros, la API ejecuta la función sobre el repositorio central y el modelo redacta la respuesta. Agiliza la atención y reduce errores de interpretación de fechas.

¿Cuáles son sus cinco componentes?
Modelo de datos: tabla auxiliar CONSULTA_ASISTENTE y funciones de lectura sobre las tablas existentes. Modelo de IA: GPT-4o vía Azure OpenAI, sin entrenamiento con datos del hotel, con un mensaje de sistema que lo limita. Lógica de integración: controlador /asistente, orquestador, filtro de datos personales y catálogo de funciones de solo lectura. Interfaz: pantalla Asistente en la app. Infraestructura: clave y punto de conexión en la configuración segura de la API, HTTPS, límites de consumo.

¿Puede el asistente crear una reserva? ¿Por qué no?
No. Solo accede a funciones de lectura; la confirmación es siempre una transacción ejecutada por el recepcionista desde la pantalla habitual, con verificación y bloqueo. Así la integridad no depende de un modelo probabilístico.

¿Qué pasa con los datos personales de los huéspedes?
No se envían al servicio externo documentos ni datos de contacto; Azure OpenAI no usa los datos enviados para entrenar; cada consulta queda registrada; y el tratamiento sigue la Ley 1581 de 2012 de protección de datos personales.

¿Por qué Azure OpenAI y no otro proveedor?
Por coherencia con la pila Microsoft (misma autenticación y despliegue), por la llamada a funciones y por la política de no entrenamiento con datos del cliente. Un proveedor equivalente podría sustituirlo sin cambiar la arquitectura, porque el servicio se encapsula en la lógica de integración.

Prototipo, evidencia y continuidad

¿Qué está implementado y qué no?
Implementado: la app .NET MAUI con la pantalla de inicio y la navegación (repositorio), los scripts SQL de creación, transacción y datos, el documento maestro completo y el prototipo interactivo de las pantallas transaccionales y del asistente, que simula la API. No implementado: la API REST, la conexión app-API y el servicio de IA real.

¿Cuáles son los próximos pasos?
Construir la API con la transacción de confirmación, conectar la app empezando por CU-01, CU-03 y CU-04, ejecutar las pruebas de aceptación de la Tabla 3 incluida la prueba de concurrencia, decidir con pruebas entre bloqueo pesimista y optimista para los cambios de estado, y evaluar la conexión del asistente.

¿Qué dificultades tuvieron?
Instalar SQL Server (se resolvió con la edición gratuita 2025 Express/Developer mediante instalación personalizada); mantener las cardinalidades idénticas entre clases, MER y modelo relacional (se corrigió la asociación Usuario–Bitácora que faltaba en el diagrama de clases); y dosificar el nivel de detalle técnico en una fase de análisis y diseño.

¿Cómo garantizan que el trabajo es propio?
El documento cita todas las fuentes en APA 7 (Elmasri y Navathe, Silberschatz, Haerder y Reuter, Gray y Reuter, Fielding, Chen, ISO/IEC 25010, documentación de Microsoft, Ley 1581); los diagramas y las pantallas son de elaboración propia y sus fuentes (PlantUML, SVG, HTML) están en el repositorio con historial de commits desde la S7.

Materiales: Presentación (HTML y PDF), Prototipo Interactivo (HTML), Ficha de Auditoría y Ficha de Requerimientos (PDF), script 04_demo_concurrencia_auditoria.sql, documento maestro Fase II y III (PDF), repositorio github.com/gustavo8015/HotelReservas.